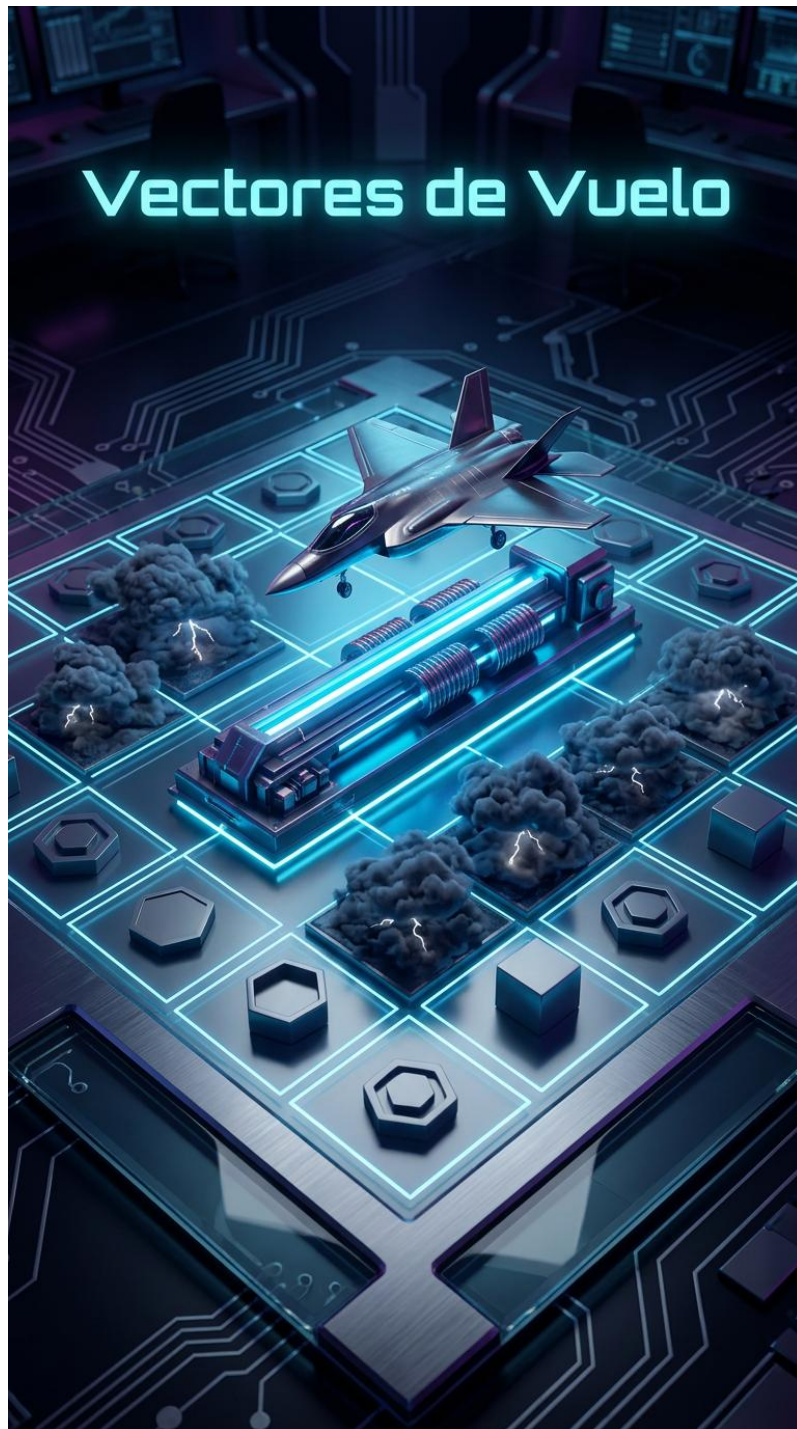


Proyecto “Vectores de Vuelo”

(Fundamentos de la Inteligencia Artificial)

FASE 1, 2 y 3



Diego Fiz García

Índice:

- **Fase 1:**
 - Descripción General del Juego
 - Elementos del Juego
 - Reglas del Juego
 - Objetivo del Sistema Basado en Reglas
 - Ejemplo de Partida

- **Fase 2:**
 - Base de Hechos (BH)
 - Base de Reglas (BR)
 - Motor de Inferencia (MI)

- **Fase 3:**
 - Código HTML

Fase 1:

1. Descripción general del juego

Este proyecto consiste en diseñar y simular un juego propio al que he llamado “Vectores de Vuelo”.

Es un juego competitivo, de información perfecta y por turnos, pensado solo para 2 jugadores. El tablero es una cuadrícula de 6x6 casillas, parecida a un tablero pequeño, y cada jugador controla un único Avión Experimental. El objetivo del juego es llevar mi avión desde mi lado del tablero hasta la base rival antes de que el otro jugador haga lo mismo conmigo.

La idea del juego mezcla dos cosas: por un lado, bloquear el avance del rival, y por otro, aprovechar movimientos automáticos para avanzar más rápido. En cada turno solo se puede hacer una acción, y esa acción puede ser mover el avión, colocar una tormenta o colocar una catapulta.

Si el avión entra en una casilla donde hay una catapulta, se activa la parte más importante del juego, el “vuelo automático”. En ese momento, el jugador deja de tomar decisiones y el sistema pasa a mover el avión por sí solo siguiendo la misma dirección en la que entró a la catapulta. Si iba hacia arriba, seguirá hacia arriba; si iba hacia la derecha, seguirá hacia la derecha.

Mientras el avión está volando, puede pasar por encima de las tormentas y también por encima del avión rival sin chocar. El vuelo sigue avanzando casilla a casilla hasta que encuentra la primera casilla libre donde puede aterrizar. Además, si esa casilla en la que aterriza tiene otra catapulta, el avión vuelve a despegar inmediatamente y sigue avanzando en la misma dirección.

Por eso, aunque el juego es fácil de entender al verlo, por detrás tiene una parte lógica bastante interesante, porque combina decisiones manuales con una resolución automática controlada por el Motor de Inferencia.

2. Elementos del juego

a. Jugadores:

En la partida participan dos jugadores: Jugador 1 y Jugador 2. Los dos juegan de forma alterna durante toda la partida.

b. Tablero o entorno:

Se utiliza una cuadrícula de 6x6 que representa un espacio aéreo en disputa. Cada casilla se identifica con unas coordenadas (X, Y) , donde tanto X como Y toman valores entre 1 y 6.

c. Elementos o fichas:

El juego está formado por los siguientes elementos:

- 1 Avión Experimental: La ficha principal que debe cruzar el tablero.
- Catapultas de Lanzamiento: Fichas tecnológicas que permiten a los aviones despegar y volar, y que sirven para activar el vuelo automático.
- Nubes de Tormenta: Fichas de obstáculos climáticos que bloquean el avance por tierra, y que actúan como obstáculos mientras el avión se mueve por tierra.

d. Estado Inicial:

- El estado inicial del juego es el siguiente:
 - o El avión del Jugador 1 empieza en la coordenada (3,1).
 - o El avión del Jugador 2 empieza en la coordenada (4,6).
 - o El tablero empieza vacío, sin catapultas ni tormentas.
 - o El primer turno se decide de forma aleatoria.
- La condición de victoria también está fijada desde el principio:
 - o El Jugador 1 gana si consigue llegar a cualquier casilla con Y=6.
 - o El Jugador 2 gana si consigue llegar a cualquier casilla con Y=1.

3. Reglas del juego

1) Turnos:

La partida se desarrolla por turnos alternos. En cada turno, el jugador que tiene el turno activo debe hacer exactamente una acción manual.

Después de esa acción, pueden pasar dos cosas:

- Que el turno termine directamente.
- O que se active una fase automática si el avión entra en una catapulta.

2) Acciones posibles (Fase Manual):

- Rodar (Mover por tierra): El jugador mueve su Avión exactamente 1 casilla de forma ortogonal, es decir, Norte, Sur, Este u Oeste. No se puede Rodar hacia una casilla que contenga una Nube de Tormenta o al avión rival.
- Instalar Catapulta: El jugador coloca una Catapulta en cualquier casilla vacía del tablero.
- Generar Tormenta: El jugador coloca una Nube de Tormenta en cualquier casilla vacía del tablero para crear un muro.

3) Restricciones y Resolución Automática (Fase de Vuelo):

- Rodar: Si el jugador decide mover su avión, solo puede avanzar una casilla. Ese movimiento tiene que ser ortogonal, no diagonal. Además, no se puede rodar hacia una casilla en la que haya una tormenta o donde esté el avión rival.
- Despegue: Si al hacer la acción de rodar el avión entra en una casilla donde hay una catapulta, el avión despegue automáticamente. A partir de ahí, el sistema toma el control del movimiento.
- Vuelo de Crucero: Mientras está en el aire, el Avión pasa por encima de las Nubes de Tormenta y del avión rival sin chocar. Sigue avanzando en línea recta de forma automática.
- Aterrizaje: El Avión aterriza obligatoriamente en la primera casilla de su trayectoria que NO sea una Nube de Tormenta ni el Avión rival.
- Combos de Vuelo: Si la casilla en la que aterriza el avión resulta ser *otra* Catapulta, vuelve a despegar instantáneamente, encadenando múltiples vuelos en un solo turno.

4) Fin del Juego:

- La partida finaliza inmediatamente cuando se cumple una de estas dos condiciones:
 - El Jugador 1 sitúa su Avión en la fila Y=6, resultando ganador.
 - El Jugador 2 sitúa su Avión en la fila Y=1, resultando ganador.
- En ese momento, el jugador correspondiente gana la partida.

Fase 2:

SBR = Sistema Basado en Reglas

BH = Base de Hechos

BR = Base de Reglas

MI = Motor de Inferencia

SBR = {BH, BR, MI}

1. Base de Hechos (BH):

La Base de Hechos representa el estado del juego en cada momento. En ella se guardan:

- La posición de los aviones,
- La posición de las catapultas,
- La posición de las tormentas,
- El turno activo,
- Los indicadores de fin de partida,
- Un hecho temporal que indica si el avión está en vuelo y en qué dirección se está moviendo.

1-. Clases del dominio:

CLASE Avion(jugador, x, y)

CLASE Catapulta(x, y)

CLASE Tormenta(x, y)

CLASE AccionRodar(jugador, direccion)

CLASE AccionCatapulta(jugador, x, y)

CLASE AccionTormenta(jugador, x, y)

CLASE VueloActivo(jugador, direccion)

CLASE Turno(jugador)

CLASE Fin()

CLASE Ganador(jugador)Estado inicial (MT0)

2-. Estado Inicial (MT0):

MT0 = {

Avion(J1, 3, 1)

Avion(J2, 4, 6)

Turno(J1)

}

2. Base de Reglas (BR):

Para definir las reglas, supongo una función auxiliar llamada Siguiente(X,Y,DIR), que calcula cuál es la casilla siguiente dependiendo de la dirección en la que se mueve el avión.

2.1 R1 - Rodar a casilla vacía:

El Si es mi turno, quiero rodar y la casilla de destino está libre, el avión se mueve a esa casilla y el turno pasa al otro jugador.

R1:

SI Turno(J1) Y AccionRodar(J1, DIR)
Y Avion(J1, X, Y) CON Siguiente(X,Y,DIR) = (Xd,Yd)
Y NO Tormenta(Xd,Yd) Y NO Catapulta(Xd,Yd) Y NO Avion(J2,Xd,Yd) ENTONCES
modificar(Avion(J1, Xd, Yd))
borrar(AccionRodar(J1, DIR))
borrar(Turno(J1))
crear(Turno(J2))

2.2 R2 - Rodar hacia Catapulta (Despegue) :

Si al rodar entro en una catapulta, el avión se mueve a esa casilla y se crea el hecho temporal de vuelo.

R2:

SI Turno(J1) Y AccionRodar(J1, DIR)
Y Avion(J1, X, Y) CON Siguiente(X,Y,DIR) = (Xd,Yd)
Y Catapulta(Xd,Yd) ENTONCES
modificar(Avion(J1, Xd, Yd))
crear(VueloActivo(J1, DIR))
borrar(AccionRodar(J1, DIR))

2.3 R3 - Fase Automática (Sobrevolar Obstáculos):

Si el avión está en vuelo y la siguiente casilla tiene una tormenta o el avión rival, el avión sigue avanzando.

R3:

SI VueloActivo(J1, DIR) Y Avion(J1, X, Y) CON Siguiente(X,Y,DIR) = (Xd,Yd)
Y (Tormenta(Xd,Yd) O Avion(J2,Xd,Yd)) ENTONCES
modificar(Avion(J1, Xd, Yd))

2.4 R4 - Fase Automática: Aterrizaje en Catapulta (Combo) :

Si el avión está en vuelo y llega a una catapulta, se mueve a esa casilla y continúa el vuelo.

R4:

SI VueloActivo(J1, DIR) Y Avion(J1, X, Y) CON Siguiente(X,Y,DIR) = (Xd,Yd)
Y Catapulta(Xd,Yd) ENTONCES
modificar(Avion(J1, Xd, Yd))

2.5 R5 - Fase Automática (Aterrizaje en Casilla Vacía):

Si el avión está en vuelo y la siguiente casilla está libre, aterriza ahí, se elimina el hecho de vuelo y el turno pasa al otro jugador.

R5:

SI VueloActivo(J1, DIR) Y Avion(J1, X, Y) CON Siguiente(X,Y,DIR) = (Xd,Yd)
Y NO Tormenta(Xd,Yd) Y NO Catapulta(Xd,Yd) Y NO Avion(J2,Xd,Yd) ENTONCES
modificar(Avion(J1, Xd, Yd))
borrar(VueloActivo(J1, DIR))
borrar(Turno(J1))
crear(Turno(J2))

2.6 R6 Acciones de Instalación (Colocar Tormentas) :

Si es mi turno y quiero poner una tormenta en una casilla libre, se crea esa tormenta y el turno pasa al otro jugador.

R6:

SI Turno(J1) Y AccionTormenta(J1, X, Y)
Y NO Tormenta(X,Y) Y NO Catapulta(X,Y) Y NO Avion(J1, X, Y) Y NO Avion(J2, X, Y)
ENTONCES
crear(Tormenta(X,Y))
borrar(AccionTormenta(J1, X, Y))
borrar(Turno(J1))
crear(Turno(J2))

2.7 R7 Acciones de Instalación (Colocar Catapultas) :

Si es mi turno y quiero poner una catapulta en una casilla libre, se crea esa catapulta y el turno pasa al otro jugador.

R7:

SI Turno(J1) Y AccionCatapulta(J1, X, Y)
Y NO Tormenta(X,Y) Y NO Catapulta(X,Y) Y NO Avion(J1, X, Y) Y NO Avion(J2, X, Y)
ENTONCES
crear(Catapulta(X,Y))
borrar(AccionCatapulta(J1, X, Y))

borrar(Turno(J1))
crear(Turno(J2))

2.8 R8 – Victoria (Jugador 1):

Si el avión del Jugador 1 llega a una casilla con $Y = 6$, el sistema crea el hecho de ganador y finaliza la partida.

R8:

SI Avion(J1, X, 6) ENTONCES
crear(Ganador(J1))
crear(Fin())

2.9 R9 – Victoria (Jugador 2):

Si el avión del Jugador 2 llega a una casilla con $Y = 1$, el sistema crea el hecho de ganador y finaliza la partida.

R9:

SI Avion(J2, X, 1) ENTONCES
crear(Ganador(J2))
crear(Fin())

3. Motor de Inferencia (MI):

- La estrategia elegida para el sistema es el “encadenamiento hacia adelante”, ya que el juego va evolucionando a partir de un estado inicial y el sistema va aplicando reglas según lo que haya en la Base de Hechos en cada momento.

- Como estrategia de control, utilizo “prioridad y novedad”:

La prioridad es importante porque las reglas de victoria tienen que ejecutarse antes que cualquier otra, y también porque las reglas de vuelo automático deben resolverse antes de devolver el turno al siguiente jugador.

- Esto permite que, cuando se crea la función de “Vuelo Activo”, el sistema siga resolviendo el movimiento automáticamente hasta que el avión aterrice o hasta que se cumpla una condición de victoria. De esta forma, el comportamiento del juego queda bien definido tanto en la parte manual como en la parte automática.

Fase 3:

1. Programación del Juego y código en lenguaje HTML:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/4.7.0/css/font-
awesome.min.css">
  <style type="text/css">
    body {
      height: 100vh;
      width: 100vw;
      display: flex;
      flex-direction: column;
      justify-content: center;
      align-items: center;
    }

    #info {
      padding: 1rem;
      display: flex;
      flex-direction: row;
      gap: 2rem;
    }

    #board {
      display: flex;
      flex-direction: column;
      gap: 5px;
      padding: 5px;
      background-color: #000;
      width: fit-content;
    }

    .row {
      display: flex;
      flex-direction: row;
      gap: 5px;
    }

    .buttons {
      padding: 1rem;
      display: flex;
      flex-direction: row;
      gap: 2rem;
    }
```

```

        .cell {
            height: 60px;
            width: 60px;
            font-size: 30px;
            display: flex;
            justify-content: center;
            align-items: center;
            background-color: #FFF;
        }

        .cell:hover {
            background-color: #F00;
            transition: 0.5s;
            cursor: pointer;
        }

        .previousCell {
            background-color: #FAA;
        }
    </style>
</head>
<body>

<h3 id="info"></h3>

<div id="board"></div>

<div class="buttons">
    <button onclick="N()">
        <i class="fa fa-arrow-up"></i> (N)
    </button>

    <button onclick="S()">
        <i class="fa fa-arrow-down"></i> (S)
    </button>

    <button onclick="E()">
        <i class="fa fa-arrow-right"></i> (E)
    </button>

    <button onclick="O()">
        <i class="fa fa-arrow-left"></i> (O)
    </button>
</div>

<div class="buttons">
    <button onclick="tormenta()">
        <i class="fa fa-cloud"></i> TORMENTA
    </button>

```

```

    <button onclick="catapulta()">
        <i class="fa fa-rocket"></i> CATAPULTA
    </button>
</div>

</body>

<script>

//-----
// BASE DE HECHOS (BH)
//-----
let estado = {
    aviones: [
        { x: 3, y: 1, jugador: "J1" },
        { x: 4, y: 6, jugador: "J2" }
    ],
    catapultas: [],
    tormentas: [],
    turno: "J1",
    vueloActivo: null,
    fin: false,
    ganador: null
};

//-----
// Funciones auxiliares
//-----
function movimiento(dir, x, y) {
    if (dir === "N") return { x, y: y - 1 };
    if (dir === "S") return { x, y: y + 1 };
    if (dir === "E") return { x: x + 1, y };
    if (dir === "O") return { x: x - 1, y };
    return { x, y };
}

function dentro(x, y) {
    return x >= 1 && x <= 6 && y >= 1 && y <=6;
}

function contieneCoordenadas(arr, x, y) {
    let i = 0;
    while (i < arr.length) {
        if (arr[i].x === x && arr[i].y === y) {
            return true;
        }
        i++;
    }
    return false;
}

```

```

}

function cambioTurno() {
  if (estado.turno === "J1") estado.turno = "J2";
  else if (estado.turno === "J2") estado.turno = "J1";
}

//-----
// Reglas (BR)
//-----

function R1(dir) { // Rodar a casilla vacía
  if (estado.vueloActivo || estado.fin) return false;
  let avion = estado.aviones.find((a) => a.jugador === estado.turno);
  const nuevaPos = movimiento(dir, avion.x, avion.y);

  if (!dentro(nuevaPos.x, nuevaPos.y)) return false;
  if (contieneCoordenadas(estado.tormentas, nuevaPos.x, nuevaPos.y))
return false;
  if (contieneCoordenadas(estado.catapultas, nuevaPos.x, nuevaPos.y))
return false;

  let otroAvion = estado.aviones.find((a) => a.x === nuevaPos.x && a.y
=== nuevaPos.y);
  if (otroAvion) return false;

  avion.x = nuevaPos.x;
  avion.y = nuevaPos.y;
  cambioTurno();
  return true;
}

function R2(dir) { // Rodar hacia Catapulta (Despegue)
  if (estado.vueloActivo || estado.fin) return false;
  let avion = estado.aviones.find((a) => a.jugador === estado.turno);
  const nuevaPos = movimiento(dir, avion.x, avion.y);

  if (!dentro(nuevaPos.x, nuevaPos.y)) return false;
  if (!contieneCoordenadas(estado.catapultas, nuevaPos.x, nuevaPos.y))
return false;

  avion.x = nuevaPos.x;
  avion.y = nuevaPos.y;
  estado.vueloActivo = { jugador: estado.turno, direccion: dir };
  return true;
}

function R3() { // Fase Automática (Sobrevolar Obstáculos)
  if (!estado.vueloActivo) return false;

```

```

    let avion = estado.aviones.find((a) => a.jugador ===
estado.vueloActivo.jugador);
    const nuevaPos = movimiento(estado.vueloActivo.direccion, avion.x,
avion.y);

    if (!dentro(nuevaPos.x, nuevaPos.y)) return false;

    let hayObstaculo = contieneCoordenadas(estado.tormentas, nuevaPos.x,
nuevaPos.y) ||
                    estado.aviones.some(a => a.x === nuevaPos.x && a.y
=== nuevaPos.y);

    if (hayObstaculo) {
        avion.x = nuevaPos.x;
        avion.y = nuevaPos.y;
        return true;
    }
    return false;
}

function R4() { // Fase Automática: Aterrizaje en Catapulta (Combo)
    if (!estado.vueloActivo) return false;
    let avion = estado.aviones.find((a) => a.jugador ===
estado.vueloActivo.jugador);
    const nuevaPos = movimiento(estado.vueloActivo.direccion, avion.x,
avion.y);

    if (!dentro(nuevaPos.x, nuevaPos.y)) return false;

    if (contieneCoordenadas(estado.catapultas, nuevaPos.x, nuevaPos.y)) {
        avion.x = nuevaPos.x;
        avion.y = nuevaPos.y;
        return true;
    }
    return false;
}

function R5() { // Fase Automática (Aterrizaje en Casilla Vacía)
    if (!estado.vueloActivo) return false;
    let avion = estado.aviones.find((a) => a.jugador ===
estado.vueloActivo.jugador);
    const nuevaPos = movimiento(estado.vueloActivo.direccion, avion.x,
avion.y);

    if (!dentro(nuevaPos.x, nuevaPos.y)) {
        // Fuera de tablero durante el vuelo: aterrizo forzosamente en el
borde (o pierde el vuelo)
        // Para simplificar, paramos el vuelo y cambiamos turno.
        estado.vueloActivo = null;
    }
}

```

```

        cambioTurno();
        return true;
    }

    let hayObstaculo = contieneCoordenadas(estado.tormentas, nuevaPos.x,
nuevaPos.y) ||
        estado.aviones.some(a => a.x === nuevaPos.x && a.y
=== nuevaPos.y);
    let hayCatapulta = contieneCoordenadas(estado.catapultas, nuevaPos.x,
nuevaPos.y);

    if (!hayObstaculo && !hayCatapulta) {
        avion.x = nuevaPos.x;
        avion.y = nuevaPos.y;
        estado.vueloActivo = null;
        cambioTurno();
        return true;
    }
    return false;
}

function R6(nuevaPos) { // Colocar tormenta
    if (estado.vueloActivo || estado.fin) return false;
    if (!dentro(nuevaPos.x, nuevaPos.y)) return false;

    if (contieneCoordenadas(estado.aviones, nuevaPos.x, nuevaPos.y))
return false;
    if (contieneCoordenadas(estado.tormentas, nuevaPos.x, nuevaPos.y))
return false;
    if (contieneCoordenadas(estado.catapultas, nuevaPos.x, nuevaPos.y))
return false;

    estado.tormentas.push(nuevaPos);
    cambioTurno();
    return true;
}

function R7(nuevaPos) { // Colocar catapulta (Simple, no teletransporte)
    if (estado.vueloActivo || estado.fin) return false;
    if (!dentro(nuevaPos.x, nuevaPos.y)) return false;

    if (contieneCoordenadas(estado.aviones, nuevaPos.x, nuevaPos.y))
return false;
    if (contieneCoordenadas(estado.tormentas, nuevaPos.x, nuevaPos.y))
return false;
    if (contieneCoordenadas(estado.catapultas, nuevaPos.x, nuevaPos.y))
return false;

    estado.catapultas.push(nuevaPos);

```

```

    cambioTurno();
    return true;
}

function R8() { // Victoria de J1
    let avion = estado.aviones.find((a) => a.jugador === "J1");
    if (avion.y === 6) {
        estado.fin = true;
        estado.ganador = "J1";
        return true;
    }
    return false;
}

function R9() { // Victoria de J2
    let avion = estado.aviones.find((a) => a.jugador === "J2");
    if (avion.y === 1) {
        estado.fin = true;
        estado.ganador = "J2";
        return true;
    }
    return false;
}

//-----
// Motor de inferencia (Bucle while real)
//-----

function resolverVuelo() {
    // Bucle del motor de inferencia para la fase automática
    while (estado.vueloActivo && !estado.fin) {
        // En cada iteración del vuelo, evaluamos las reglas de vuelo por
        // prioridad
        if (R8() || R9()) break;

        let reglaAplicada = R4() || R5() || R3();
        // Primero intentamos aterrizar en catapulta, luego en vacía, y
        // por último sobrevolar

        if (!reglaAplicada) {
            // Si el avión fuera a salir del mapa o algo imprevisto
            estado.vueloActivo = null;
            cambioTurno();
            break;
        }
    }
}

function move(dir) {

```



```

    if (!estado.fin && !estado.vueloActivo) {
        if (!R1(dir)) {
            R2(dir); // Si no puede hacer movimiento simple, intenta
despegue
        }
    }

    resolverVuelo(); // Llama al bucle automático
    update();
}

function N() { move("N"); }
function S() { move("S"); }
function E() { move("E"); }
function O() { move("O"); }

let selectionMode = null;

function tormenta() { selectionMode = "T"; }
function catapulta() { selectionMode = "C"; }

function cellClicked(x, y) {
    if (estado.fin) return;

    const pos = { x: Number(x), y: Number(y) };
    if (selectionMode === "T") {
        R6(pos);
        selectionMode = null;
    } else if (selectionMode === "C") {
        R7(pos);
        selectionMode = null;
    }
    update();
}

function update() {
    if (!estado.fin) {
        R8();
        R9();
    }
    render();
}

// Render
function render() {
    let board = document.getElementById("board");
    board.innerHTML = "";
    for (var y = 1; y <= 6; y++) {
        let row = document.createElement("div");

```

```

    row.className = "row";
    for (var x = 1; x <= 6; x++) {
        let cell = document.createElement("div");
        cell.className = "cell";

        // Distintivos J1 y J2
        let avionEnCasilla = estado.aviones.find(a => a.x === x &&
a.y === y);
        if (avionEnCasilla) {
            const icon = document.createElement("i");
            icon.className = "fa fa-plane";
            if (avionEnCasilla.jugador === "J1") icon.style.color =
"blue";
            if (avionEnCasilla.jugador === "J2") icon.style.color =
"red";

            cell.appendChild(icon);
        } else if (contieneCoordenadas(estado.catapultas, x, y)) {
            const icon = document.createElement("i");
            icon.className = "fa fa-rocket";
            cell.appendChild(icon);
        } else if (contieneCoordenadas(estado.tormentas, x, y)) {
            const icon = document.createElement("i");
            icon.className = "fa fa-cloud";
            cell.appendChild(icon);
        }

        cell.setAttribute("x", x);
        cell.setAttribute("y", y);
        row.appendChild(cell);
    }
    board.appendChild(row);
}

const cells = document.getElementsByClassName("cell");
for (let i = 0; i < cells.length; i++) {
    cells[i].onclick = () => cellClicked(
        cells[i].getAttribute("x"),
        cells[i].getAttribute("y")
    );
}

if (estado.ganador) {
    document.getElementById("info").innerText = '¡Gana ' +
estado.ganador + '!';
} else {
    document.getElementById("info").innerText = 'Es el turno de ' +
estado.turno;
}
}

```

```
update();
```

```
</script>
```

```
</html>
```